

Ракитский К.С.

(#####)

**ПРИМЕР ПРОГРАММНОЙ
ДОКУМЕНТАЦИИ НА КАЛЬКУЛЯТОР РОСТА
РАСТЕНИЙ,
РАЗРАБОТАННЫЙ НА ОСНОВЕ
ЯЗЫКА PYTHON С
ИСПОЛЬЗОВАНИЕМ TKINTER**

**ПЕТРОПАВЛОВСК-
КАМЧАТСКИЙ**

2026

СОДЕРЖАНИЕ

1.	ВВЕДЕНИЕ	3
2.	ТЕХНИЧЕСКОЕ ЗАДАНИЕ НА КАЛЬКУЛЯТОР ДЛЯ РАСТЕНИЙ	4
2.1	ВВЕДЕНИЕ	4
2.2	НАЗНАЧЕНИЕ РАЗРАБОТКИ	4
3.	ТРЕБОВАНИЯ К ПРОГРАММНОМУ ОБЕСПЕЧЕНИЮ	4
4.	ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ	6
5.	ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ	7
6.	СТАДИИ И ЭТАПЫ РАЗРАБОТКИ	7
7.	ОБЩАЯ ЛОГИЧЕСКАЯ СХЕМА РАЗРАБОТАННОЙ ПРОГРАММЫ	8
7.1	Общая структурная схема программ, разрабатываемых с использованием Python	8
7.2	Порядок взаимодействия частей калькулятора	9
8.	ПЕРЕЧЕНЬ И НУМЕРАЦИЯ СОБЫТИЙ	10
9.	ПЕРЕЧЕНЬ И НУМЕРАЦИЯ ВХОДНЫХ ПЕРЕМЕННЫХ	12
10.	ПЕРЕЧЕНЬ И НУМЕРАЦИЯ ВЫХОДНЫХ ВОЗДЕЙСТВИЙ	13
11.	КАК ПОЛЬЗОВАТЬСЯ ВАШЕЙ ПРОГРАММОЙ	14
12.	Описание логических зависимостей программы	15
13.	ЛИСТИНГ	16

1. ВВЕДЕНИЕ

Предлагаемый пример проектной документации реально разработанного и внедренного программного обеспечения демонстрирует результат применения Python с использованием Tkinter, описанной или разработанной на основе проектного задания по дисциплине “Разработка программных модулей”

Рассматриваемое программное обеспечение разработано на основе выданного Преподавателем технического задания, которое также приведено ниже с незначительными изменениями.

Разработанная программная система расчёта роста растений состоит из двух взаимосвязанных компонентов:

- модуля расчёта циклов роста (реализован в функции `calculate_plants()`), который работает как отдельный вычислительный блок. Он принимает входные данные (выбранное растение и целевую дату), выполняет все необходимые расчёты (определяет количество полных циклов роста и оставшиеся дни) и формирует итоговый текстовый результат для отображения.

- модуля визуализации роста (реализован в функции `draw_plants()`), который интегрирован в графический интерфейс программы в виде вспомогательного инструмента. Его задача — наглядно отобразить результаты расчётов в графической форме на холсте Canvas.

Модуль расчёта и модуль визуализации взаимодействуют асинхронно через набор общих переменных и параметров, передаваемых между функциями:

`full_cycles` — количество полностью выросших растений;

`remaining_days` — количество дней, прошедших с момента последнего сбора урожая или до первого урожая;

`plant_name` — название выбранного растения, определяющее стиль отрисовки.

Фрагмент логики отрисовки конкретного растения (например, подсолнечника, томата, моркови) также реализован как условный автомат внутри функции `draw_plants()`. В зависимости от значения `plant_name` программа выбирает соответствующий набор графических примитивов (`create_oval`, `create_polygon` и т.д.) для отрисовки иконки растения.

Отметим, что рассмотренный пример является достаточно «простым» в рамках классической архитектуры GUI-приложений, так как в нём компоненты взаимодействуют только через передачу данных (параметров функции), а не через более сложные механизмы.

2. ТЕХНИЧЕСКОЕ ЗАДАНИЕ НА КАЛЬКУЛЯТОР ДЛЯ РАСТЕНИЙ

2.1 ВВЕДЕНИЕ

Наименование разработки: «Калькулятор роста растений» (далее — «калькулятор»). Калькулятор представляет собой автономное графическое приложение на базе библиотеки tkinter (Python), предназначенное для расчёта циклов роста садовых и огородных культур с визуализацией результатов.

2.2 НАЗНАЧЕНИЕ РАЗРАБОТКИ

Калькулятор роста растений предназначен для расчёта количества полных циклов роста выбранных растений в заданный период на основе длительности роста для каждого вида, а также для определения количества оставшихся дней до следующего урожая либо прошедших с момента последнего сбора. Программа отображает результаты расчёта в текстовой форме — с указанием количества растений и соответствующих сроков, — и визуализирует их на графическом холсте: показывает иконки растений (не более 10 штук) и индикатор прогресса для текущего цикла роста. Кроме того, калькулятор информирует пользователя об ошибках при некорректном вводе данных — например, если введены нечисловые значения или указана дата в прошлом. Взаимодействие с пользователем осуществляется через графический интерфейс на базе библиотеки tkinter (Python), а расчёты выполняются с опорой на предопределённый справочник растений и их сроков роста.

3. ТРЕБОВАНИЯ К ПРОГРАММНОМУ ОБЕСПЕЧЕНИЮ

3.1. Требования к функциональным характеристикам

3.1.1. Требования к инструментальной среде

3.1.1.1 Программа предназначена для функционирования под управлением Python версии не ниже 3.13. Разработка калькулятора выполняется с использованием стандартной библиотеки tkinter (включая модуль ttk) и модуля datetime для работы с датами. Дополнительные компиляторы или специализированные инструментальные средства не требуются.

3.1.2. Требования к пользовательскому интерфейсу

Калькулятор запускается как самостоятельное графическое приложение и не требует интеграции с другими процессами для базового функционирования. Программа должна корректно отображаться в графическом разрешении не ниже 800×600 пикселей. Размер основного окна приложения составляет 450×500 px.

Элементы интерфейса должны соответствовать следующим правилам оформления:

кнопки в активном состоянии имеют стандартный вид виджета tkinter.Button (стиль по умолчанию);

при наведении курсора на кнопку визуальная реакция отсутствует (анимация не предусмотрена);

текстовые метки и поля ввода используют стандартные стили tkinter, цвет текста — чёрный, фон — белый;

сообщения об ошибках выводятся красным цветом, информационные сообщения — чёрным, вспомогательные подписи — серым;

холст визуализации (Canvas) имеет белый фон и тонкую рамку для чёткого выделения области отображения.

3.1.3. Требования к реализуемым функциям

3.1.1.1 Предоставление пользователю выпадающего списка растений для выбора (с использованием виджета `tk.Combobox`). Перечень растений берётся из внутреннего справочника `plants_data`.

3.1.3.2 Приём от пользователя целевой даты для расчёта (день, месяц, год) через отдельные поля ввода (`tk.Entry`).

3.1.3.3 Автоматический подхват текущей даты при запуске программы: поля ввода даты заполняются текущими значениями дня, месяца и года.

3.1.3.4 Выполнение расчётов при нажатии кнопки «Рассчитать рост»:

проверка корректности введённых данных (числовой формат, существование даты);

валидация целевой даты (должна быть строго в будущем относительно текущей даты);

расчёт общего количества дней между текущей и целевой датой;

вычисление количества полных циклов роста растений на основе длительности роста выбранного вида;

определение количества оставшихся дней до следующего урожая (или после последнего урожая).

3.1.3.5 Отображение текстового результата расчёта в специальном поле (`tk.Label`), включая:

количество растений, успевших вырасти за выбранный период;

количество дней, прошедших с момента последнего урожая или оставшихся до первого урожая;

сообщения об ошибках при некорректном вводе данных.

3.1.3.6 Визуализация результатов на холсте Canvas (300×150 px):

отображение иконок растений (максимум 10 штук);

использование разных графических примитивов (`create_oval`, `create_polygon` и т.д.) для отображения разных видов растений (подсолнечник, томат, морковь и т.п.);

визуализация стадии роста текущего растения (если цикл ещё не завершён) с помощью индикатора прогресса.

3.1.3.7 Очистка холста и обновление отображения при каждом новом расчёте.

3.1.3.8 Обработка ошибок и информирование пользователя:

при вводе нечисловых значений — сообщение «Ошибка: введите корректные числа!» (красный цвет);

при выборе даты в прошлом — сообщение «Ошибка: выберите будущую дату!» (красный цвет);

при отсутствии данных для отображения (0 растений и 0 дней) — текст «Нет данных для отображения» (серый цвет) по центру холста.

3.1.3.9 Поддержка внутреннего справочника растений и их сроков роста в виде словаря `plants_data`, где ключ — название растения (строка), значение — длительность роста в днях (целое число).

3.2 Требования к составу и параметрам технических средств

3.2.1 Калькулятор роста растений предназначен для функционирования на стандартных персональных компьютерах (ПК) с базовой конфигурацией, типичной для офисных и домашних систем. Специальные требования к аппаратной платформе отсутствуют.

3.2.2 Программа должна корректно отображаться и функционировать при графическом разрешении экрана не ниже 800×600 пикселей. Оптимальное разрешение — от 1024×768 пикселей и выше.

3.2.3 Параметры интерфейса и визуальные требования
размер основного окна приложения: 450×500 px;
холст визуализации (Canvas) имеет фиксированный размер: 300×150 px, белый фон, тонкая рамка для чёткого выделения области отображения;

элементы интерфейса должны корректно отображаться при стандартном DPI (96 точек/дюйм) без необходимости масштабирования;

поддержка отображения кириллических символов во всех текстовых элементах интерфейса (заголовки, метки, сообщения об ошибках, названия растений).

3.2.3 операционная система: любая ОС с поддержкой Python 3.13+ (Windows, macOS, Linux);

язык программирования: Python версии не ниже 3.13;

обязательные библиотеки:

`tkinter` (включая модуль `ttk`) — для построения графического интерфейса;

`datetime` — для работы с датами и временными интервалами;

3.3 Специальные требования

Специальные требования не предъявляются.

4. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ

По окончании проектирования Разработчик предоставляет перподавателю следующие документы:

Перечни событий, входных переменных и выходных воздействий, участвующих в работе подсистемы.

Схемы связей и графы переходов конечных автоматов, специфицирующих поведение подсистемы.

Протоколы работы подсистемы для различных режимов работы.

5. ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ

Требования к технико-экономическим показателям не предъявляются.

6. СТАДИИ И ЭТАПЫ РАЗРАБОТКИ

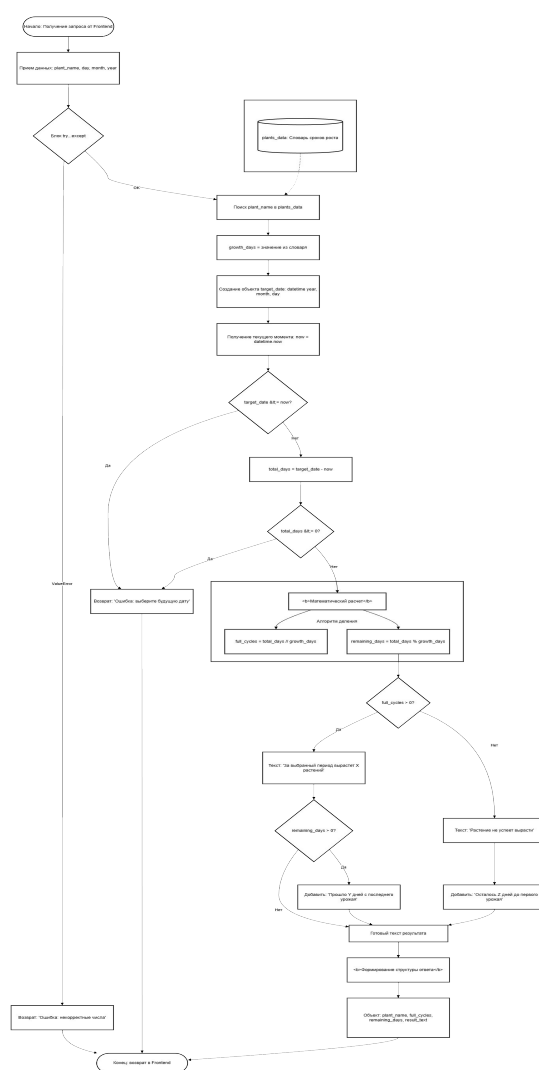
- 1) Анализ требований и проектирование интерфейса:
- 2) Реализация базового функционала:

- 3) Программирование логики расчётов:

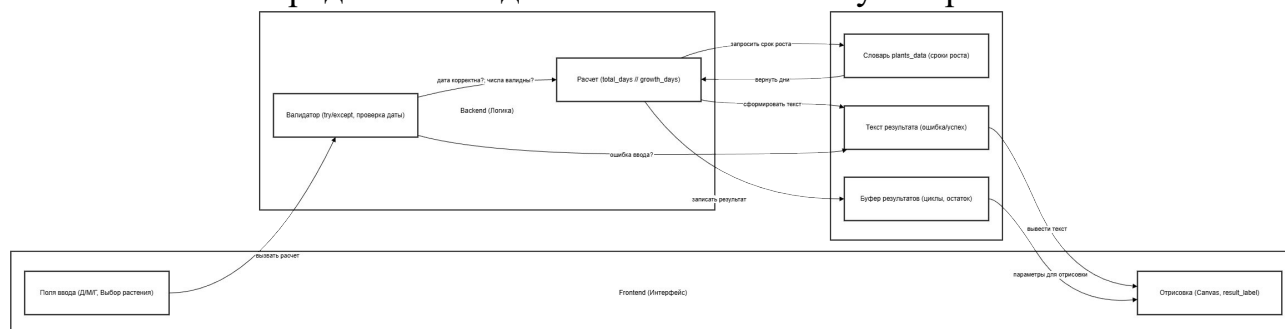
- 4) Разработка механизма визуализации:

- 5) Отладка и тестирование:

Frontend



7.2 Порядок взаимодействия частей калькулятора



8. ПЕРЕЧЕНЬ И НУМЕРАЦИЯ СОБЫТИЙ

Описание

Для калькулятора роста растений

6-15	Инициализация справочника растений и сроков их роста (в днях)	
17	Запуск расчёта количества циклов роста и оставшихся на основе введённых данных	дней
19	Получение выбранного названия растения из выпадающего списка (Combobox)	
20	Извлечение длительности роста выбранного растения из справочника	
22-24	Преобразование введённых пользователем значений дня, месяца и года в числовой формат	
26	Создание объекта целевой даты на основе пользовательского ввода	
27	Получение текущей даты и времени	
29-32	Валидация целевой даты: должна быть строго в будущем относительно текущей даты	
34	Вычисление временного интервала между текущей и целевой датой	
37-40	Извлечение общего количества дней из временного интервала	
42	Расчёт количества полных циклов роста растения	
43	Определение количества оставшихся дней до следующего урожая или после последнего	
45-50	Составление текстового сообщения с результатами расчёта для отображения пользователю	
52	Вывод текстового результата расчёта в интерфейсе (с чёрным цветом текста)	
54	Запуск функции визуализации результатов на холсте Canvas	
60	Функция отрисовки иконок растений и индикатора прогресса на холсте	
70-71	Ограничение количества отображаемых растений максимум 10 штуками	
73-84	Отрисовка иконок растений на холсте с учётом типа выбранного растения (подсолнечник, томат, морковь и т. д.)	
86-91	Отрисовка индикатора прогресса для текущего (незавершённого) цикла роста	
93-95	Создание главного окна приложения Установка заголовка окна Задание размеров окна приложения (450×500 px)	
97-98	Добавление заголовка интерфейса («Рост растений») с увеличенным шрифтом	
100-105	Настройка выпадающего списка (Combobox) для выбора растения с привязкой к переменной plant_var	

110-120	Размещение и компоновка полей для ввода дня, месяца и года с соответствующими метками
122-123	Создание кнопки «Рассчитать рост» с привязкой команды <code>calculate_plants</code>
125-126	Инициализация поля для вывода результатов расчёта
128-129	Создание холста визуализации (Canvas) размером 300×150 px с белым фоном и рамкой
131-132	Добавление информационной метки о ограничении визуализации (максимум 10 растений)
134-137	Автоматическое заполнение полей ввода даты текущими значениями (день, месяц, год)
139	Запуск главного цикла обработки событий графического интерфейса

9. ПЕРЕЧЕНЬ И НУМЕРАЦИЯ ВХОДНЫХ ПЕРЕМЕННЫХ

Описание

Для калькулятора роста растений

17	calculate_plants() - Основная функция расчёта: получает входные данные, выполняет вычисления количества циклов роста и оставшихся дней, вызывает визуализацию результатов
19	plant_name - Название выбранного растения из предопределённого списка: «Подсолнечник», «Томат», «Морковь», «Салат», «Огурец», «Редис», «Петрушка», «Укроп»
22	day - День целевой даты, введённый пользователем (число от 1 до 31)
23	month - Месяц целевой даты, введённый пользователем (число от 1 до 12)
24	year - Год целевой даты, введённый пользователем (например, 2024, 2025 и т.д.)

10. ПЕРЕЧЕНЬ И НУМЕРАЦИЯ ВЫХОДНЫХ ВОЗДЕЙСТВИЙ

Описание

Результаты отработки

- | | |
|---------|---|
| 30, 38 | Вывод сообщения об ошибке «Ошибка: выберите будущую дату!» (красный текст в result_label) |
| 46 | Вывод результата расчёта (чёрный текст в result_label, например: «За выбранный период вырастет 2 растений: Подсолнечник. Прошло 15 дней с последнего выращенного растения») |
| 57 | Вывод сообщения об ошибке «Ошибка: введите корректные числа!» (красный текст в result_label) |
| 31, 58 | Очистка холста визуализации(canvas.delete("all")) |
| 73-84 | Отрисовка иконок растений на холсте canvas (до 10 штук, в зависимости от типа растения: подсолнечник, томат, морковь и т.д.) |
| 86-91 | Отрисовка индикатора прогресса для незавершённого цикла роста (вертикальная линия и голубая окружность на canvas) |
| 110-120 | Автоматическое заполнение полей ввода даты текущими значениями системы (day_entry, month_entry, year_entry) |
| 131-132 | Отображение информационной подсказки «Визуализация максимум 10 растений» (серый текст в info_label) |
| 139 | Запуск главного цикла обработки событий GUI (root.mainloop()) |

11. КАК ПОЛЬЗОВАТЬСЯ ВАШЕЙ ПРОГРАММОЙ

Шаг 1. Запуск программы

Сохраните код в файле с расширением .py.

Откройте файл в любом редакторе кода поддерживающий Python или через командную строку (В системе должен быть установлен Python 3.13+): `python plant_growth.py`.

Откроется окно программы с заголовком «Рост растений» размером 450×500 px.

Шаг 2. Выбор растения

Найдите выпадающий список с надписью «Выберите растение:».

Нажмите на список и выберите нужное растение из предложенных вариантов:

Подсолнечник (90 дней роста);

Томат (60 дней);

Морковь (70 дней);

Салат (30 дней);

Огурец (50 дней);

Редис (25 дней);

Петрушка (80 дней);

Укроп (40 дней).

Шаг 3. Ввод целевой даты

В форме под списком растений найдите три поля ввода:

«День:» — введите число месяца (1–31);

«Месяц:» — введите номер месяца (1–12);

«Год:» — введите год (например, 2024, 2025 и т.д.).

По умолчанию поля заполнены текущей датой (день, месяц, год на момент запуска программы). Вы можете оставить эти значения или изменить их.

Шаг 4. Расчёт и получение результатов

Нажмите кнопку «Рассчитать рост».

Программа выполнит расчёт и отобразит результаты в двух областях:

Текстовый результат (под кнопкой) — сообщение о количестве выросших растений и оставшихся днях.

Визуализация (холст 300×150 px) — графическое отображение растений и прогресса роста.

12. Описание логических зависимостей программы

```

import tkinter as tk
from tkinter import ttk
from datetime import datetime

# 1. Данные (базовый блок)
plants_data = {
    "Подсолнечник": 90, "Томат": 60, "Морковь": 70,
    "Салат": 30, "Огурец": 50, "Редис": 25,
    "Петрушка": 80, "Укроп": 40
} # Зависимости: нет

# 2. Расчёт роста растений
def calculate_plants():
    # Зависимости: plants_data, элементы GUI (plant_var, day_entry и т.д.)
    try:
        plant_name = plant_var.get()
        growth_days = plants_data[plant_name]
        # ... считывание даты и проверка
        full_cycles = total_days // growth_days
        remaining_days = total_days % growth_days
        # ... формирование текста результата
        result_label.config(text=result_text)
        draw_plants(full_cycles, remaining_days, plant_name) # Вызов визуализации
    except ValueError:
        result_label.config(text="Ошибка: введите корректные числа!", fg="red")

# 3. Визуализация на холсте
def draw_plants(full_cycles, remaining_days, plant_name):
    # Зависимости: plants_data (для расчёта прогресса), canvas
    canvas.delete("all")
    if full_cycles == 0 and remaining_days == 0:
        canvas.create_text(...)
        return
    max_plants = min(full_cycles, 10)
    # ... отрисовка полных растений (с учётом типа)
    if remaining_days > 0 and full_cycles < 10:
        progress = remaining_days / plants_data[plant_name]
        height = 20 + (60 * progress)
        # ... отрисовка растущего растения

# 4. Создание GUI
root = tk.Tk()
# ... создание виджетов: заголовок, выбор растения, поля даты, кнопка, метка результата, холст, инфо-метка
plant_var = tk.StringVar(value="Подсолнечник")

```

```
# ... остальные элементы интерфейса
day_entry.insert(0, str(now.day)) # Текущая дата по умолчанию
# Зависимости: plants_data (для Combobox), функции calculate_plants (команда кнопки)

# 5. Главный цикл
root.mainloop() # Зависимости: root (главное окно)
```

13. ЛИСТИНГ

```
import tkinter as tk
from tkinter import ttk
from datetime import datetime, timedelta
import random

plants_data = {
    "Подсолнечник": 90,
    "Томат": 60,
    "Морковь": 70,
    "Салат": 30,
    "Огурец": 50,
    "Редис": 25,
    "Петрушка": 80,
    "Укроп": 40
}

def calculate_plants():
    try:
        plant_name = plant_var.get()
        growth_days = plants_data[plant_name]

        day = int(day_entry.get())
        month = int(month_entry.get())
        year = int(year_entry.get())

        target_date = datetime(year, month, day)
        now = datetime.now()

        if target_date <= now:
            result_label.config(text="Ошибка: выберите будущую дату!", fg="red")
            canvas.delete("all")
            return

        diff = target_date - now
        total_days = diff.days

        if total_days <= 0:
            result_label.config(text="Ошибка: выберите будущую дату!", fg="red")
```



```

        canvas.create_rectangle(x-3, canvas_height-20, x+3, canvas_height-80, fill="green",
                                outline="darkgreen")
    else:
        canvas.create_oval(x-12, canvas_height-85, x+12, canvas_height-65, fill="lightgreen",
                           outline="green")

    if remaining_days > 0 and full_cycles < 10:
        x = (full_cycles + 1) * plant_width
        progress = remaining_days / plants_data[plant_name]
        height = 20 + (60 * progress)
        canvas.create_line(x, canvas_height - 20, x, canvas_height - 20 - height, width=2, fill="green")
        canvas.create_oval(x-8, canvas_height-30-height, x+8, canvas_height-10-height,
                           fill="lightblue", outline="blue")

root = tk.Tk()
root.title("Пост растений")
root.geometry("450x500")

title_label = tk.Label(root, text="Пост растений", font=("Arial", 16, "bold"))
title_label.pack(pady=10)

plant_label = tk.Label(root, text="Выберите растение:")
plant_label.pack()

plant_var = tk.StringVar(value="Подсолнечник")
plant_combo = ttk.Combobox(root, textvariable=plant_var, values=list(plants_data.keys()),
                             state="readonly")
plant_combo.pack(pady=5)

date_frame = tk.Frame(root)
date_frame.pack(pady=10)

tk.Label(date_frame, text="День:").grid(row=0, column=0, padx=5)
day_entry = tk.Entry(date_frame, width=5)
day_entry.grid(row=0, column=1, padx=5)

tk.Label(date_frame, text="Месяц:").grid(row=0, column=2, padx=5)
month_entry = tk.Entry(date_frame, width=5)
month_entry.grid(row=0, column=3, padx=5)

tk.Label(date_frame, text="Год:").grid(row=0, column=4, padx=5)
year_entry = tk.Entry(date_frame, width=8)
year_entry.grid(row=0, column=5, padx=5)

calculate_button = tk.Button(root, text="Рассчитать рост", command=calculate_plants)
calculate_button.pack(pady=10)

```

```
result_label = tk.Label(root, text="", font=("Arial", 10), justify="left", anchor="w")
result_label.pack(pady=10)

canvas = tk.Canvas(root, width=300, height=150, bg="white", relief="solid", bd=1)
canvas.pack(pady=10)

info_label = tk.Label(root, text="Визуализация максимум 10 растений", font=("Arial", 8),
fg="gray")
info_label.pack()

now = datetime.now()
day_entry.insert(0, str(now.day))
month_entry.insert(0, str(now.month))
year_entry.insert(0, str(now.year))

root.mainloop()
```